응용 8_4_1 파일

코드 업그레이드

발표자 : 노정균	2021963025

목차

1	응용 8_4_1 파일의 구조	파일의 구조가 어떻게 되어있지?
2	파일 업그레이드	기존 코드에서 뭐가 추가되었지?
3	업그레이드 총 정리	업그레이드 내역이 어떻게 되지?

응용 8_4_1 파일의 구조 (1)

```
#include <stdio.h>
#include <conio.h>
#include <windows.h>
void gotoxy(int x, int y);
void display_text(int count);
void display_question(void);
void display_result(void);
void intro_game(void);
char sol[10][11];
int main(void) {
    intro_game();
    display_question();
    printf("\n문제풀이가 모두 끝났습니다.\n");
    printf("\n\n풀이를 보려면 아무키나 누르시오.");
    getch();
    display_result();
    return 0;
}
```

[전체 흐름]

- ① 시작 안내
- ② 질문 전체 진행
- ③ 키 입력 대기
- ④ 해설 전체 진행
- ⑤ 프로그램 종료

응용 8_4_1 파일의 구조 (2)

```
void gotoxy(int x, int y)
{
    COORD Pos = {x - 1, y - 1};
    SetConsoleCursorPosition(
        GetStdHandle(STD_OUTPUT_HANDLE), Pos);
}
```

COORD Pos : Pos 라는 자료형 변수에 콘솔의 좌표를 담음

x - 1, y - 1을 하는 이유 : 사용자는 (1,1) 부터 입력 받기 때문이다.

GetStdHandle(STD_OUTPUT_HANDLE), Pos) : 해당 좌표 위치에 출력 핸들을 놓음

응용 8_4_1 파일의 구조 (3)

```
void display_text(int count) {
    gotoxy(1,2);
    printf("다음 문제를 읽고 답을 입력하시오.\n\n");
    printf("(%d번 문제) \n\n", count+1);
}
```

1. 화면을 초기화 한다.
2. printf를 모두 실행하여 문자를 출력한다.
3. 사용자가 키를 하나 누를 때까지 기다립니다. 키를 누르면 함수가 끝나고 다음 단계로 진행합니다.

응용 8_4_1 파일의 구조 (4)

```
void display_question(void) {
    FILE *fp1;
    char chr1[82];
    int line = 5, count = 0;
    char *file1 = "심리테스트질문.txt";

  fp1 = fopen(file1, "r");
  system("cls");

  while (!feof(fp1)) {
    fgets(chr1, 80, fp1);

    if (chr1[0] == 10) {
      display_text(count);
      gotoxy(1, line);
      scanf("%s", sol[count]);
      system("cls");
      line = 5;
      count++;
      }
```

```
    else {
      gotoxy(1, line++);
      printf("%s", chr1);
      }
  }
  display_text(count);
  gotoxy(1, line);
  scanf("%s", sol[count]);
  fclose(fp1);
}
```

1. 질문 파일을 연다.
2. 파일에서 한 줄씩 읽는다.
3. 빈 줄을 만나면 답을 입력받아 sol[count]에 저장한다.
4. 파일 끝에 도달하면 마지막 문제의 답을 입력받는다.
5. 파일을 닫는다

응용 8_4_1 파일의 구조 (5)

```
void display_result(void) {
 FILE *fp2;
 char chr1[82];

 int line=5, count=0;
 char *file2="심리테스트해설.txt";
 fp2=fopen(file2, "r");
 system("cls");

 while(!feof(fp2)) {
       fgets(chr1, 80, fp2);

       if (chr1[0]==10) {
             gotoxy(1, line+1);
             printf("당신의 답은 : %s\n", sol[count]);
             printf("다음문제의 해설은 아무키나 누르시오.");
             getch();
             system("cls");
             line=5;
             count++;
        }
```

```
else {
       gotoxy(1, line++);
       printf("%s", chr1);
        }
  }
 gotoxy(1, line+1);
 printf("당신의 답은 : %s\n", sol[count]);
 fclose(fp2);
}
```

1. 해설 파일을 연다.
2. 파일을 한 줄씩 읽어 화면에 출력한다.
   - ㄴ 읽은 줄이 빈 줄이 아니면 현재 행에 해설 내용을 출력
   - ㄴ 빈 줄을 만나면, 현재 순서에 해당하는 답을 함께 출력
3. 키 입력을 기다린 뒤 화면을 지우고 다음 해설을 준비한다.
4. 반복문이 끝나면 마지막 답을 출력하고 파일을 닫는다.

응용 8_4_1 파일의 구조 (6)

```
void intro_game(void) {
    system("cls");
    printf("심리테스트 게임\n\n");
    printf("제시되는 문제에 모두 답을 하고나면\n");
    printf("해설이 출력됩니다.\n\n");
    printf("아무키나 누르면 시작합니다. ");
    getch();
}
```

1. count를 전달받는다.
2. 안내 문구와 문제 번호(count + 1)를 출력한다.
3. 아무 문자를 입력해 함수를 종료시킨다.

파일 업그레이드

```
==================================================
어둡고 퀴퀴한 냄새가 나는 낡은 방이다.
앞에는 굳게 닫힌 철문이 있고,
한쪽 구석에는 낡은 책상과 비밀번호가 걸린 금고가 보인다.
방을 조사해서 탈출할 방법을 찾아야 한다.
==================================================

[ 에피소드 1 : 현재 행동을 선택하세요 ]
1. 책상 조사하기
2. 자물쇠 걸린 금고 조사하기
3. 아이템 조합하기
4. 굳게 닫힌 문 열기 (탈출 시도)
선택:
```

```
==================================================
철문을 열고 나오자 새로운 방이 나타났다.
이곳은 붉은 조명이 켜져 있고 기분 나쁜 소리가 들린다...
==================================================

[ 에피소드 2 : 현재 행동을 선택하세요 ]
1. 낡은 책장 조사하기
2. 기계 장치 조작하기
3. 아이템 조합하기
4. 어두운 통로로 진입하기 (탈출 시도)
선택:
```

```
==================================================
어두운 통로를 빠져나오자, 거대한 서버가 가동 중인 [메인 통제실]이 나타났다.
중앙에는 보안이 걸린 [메인 단말기]가 있고, 벽면에는 [전자 금고]가 있다.
이곳의 메인 시스템을 해킹하고 최종 보안문을 열어야만
비로소 바깥세상으로 나갈 수 있다...!
==================================================

[ 에피소드 3 : 최종 행동을 선택하세요 ]
1. 메인 단말기 접속 (날짜 암호 해제)
2. 서버 방화벽 우회 (방향키 미니게임)
3. 전자 금고 해킹 (숫자 야구 논리 퍼즐)
4. 최종 보안문 열기 (탈출 시도)
선택:
```

파일 업그레이드

char sol[10][11]; - (정답 저장 배열 삭제)

if (chr1[0] == 10) - (엔터키로 문제 끊어 읽기 삭제)

feof(fp1) 루프 - (파일 끝 감지 방식 변경)

display_result() - (해설 출력 함수 삭제)

scanf("%s", ...) - (문자열 대기 삭제)

파일 업그레이드 (1)

```
void set_color(int color) {
    SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE), color);
}
```

[set_color(13)]

```
▶ 책상 서랍을 열어보니 [부러진 열쇠 머리]가 있다! 획득했다.
```

[set_color(10)]

```
▶ [황금 열쇠]를 구멍에 넣고 돌리자 문이 열렸다!
▶ 방탈출에 성공했습니다!
```

[set_color(12)]

```
▶ [경고] 기계 장치를 켜자 숨겨진 시한폭탄이 작동했습니다!
▶ 남은 시간은 60초! 서둘러 방을 탈출하세요!
```

파일 업그레이드 (2)

[아이템 얻었을 때 효과음]

```
void play_item_sound() {
Beep(784, 150);
Beep(1046, 300);
}
```

[문제 틀렸을 때 효과음]

```
void play_error_sound() {
Beep(300, 300);
}
```

[폭탄 터지는 효과음]

```
void play_explosion_sound() {
Beep(150, 500);
Beep(100, 800);
}
```

[스테이지 클리어 효과음]

```
void play_clear_sound() {
Beep(523, 200);
Beep(659, 200);
Beep(784, 200);
Beep(1046, 600);
}
```

파일 업그레이드 (3)

```
▶금고에 4자리 비밀번호가 필요하다. (힌트: 일이삼사)
비밀번호 입력: 1234
▶철컥! 금고가 열리며 [열쇠 손잡이]를 획득했다!

[ 에피소드 1 : 현재 행동을 선택하세요 ]
1. 책상 조사하기
2. 자물쇠 걸린 금고 조사하기
3. 아이템 조합하기
4. 굳게 닫힌 문 열기 (탈출 시도)
선택:
```

```
▶책상에는 더 이상 쓸만한 것이 없다.

[ 에피소드 1 : 현재 행동을 선택하세요 ]
1. 책상 조사하기
2. 자물쇠 걸린 금고 조사하기
3. 아이템 조합하기
4. 굳게 닫힌 문 열기 (탈출 시도)
선택:
```

[무한 루프(Game Loop)와 상태 관리 (인벤토리) 코드]

```c
while (1) {
    printf("\n행동 선택 (1.책상조사 2.금고조사 3.조합 4.문열기): ");
    scanf("%d", &choice);
if (has_key_head == 1 && has_key_body == 1) {
    printf("▶ 두 부품을 조합해 황금 열쇠를 만들었다!\n");
    has_golden_key = 1; // 🎒 완성품 획득
    has_key_head = 0;   // 기존 재료는 소모됨 (1 -> 0으로 변경)
    has_key_body = 0;
} else {
    printf("▶ 아이템이 부족해서 조합할 수 없다.\n");
}
```

```c
else if (choice == 4) {
    if (has_golden_key == 1) { // 클리어 조건을 만족했는가?
        printf("▶ 문이 열렸다! 방탈출 성공!\n");
        return 1; // 🔄 여기서 while(1) 무한 루프가 완전히 종료됨!
```

파일 업그레이드 (4)

```
▶[경고] 기계 장치를 쳐자 숨겨진 시한폭탄이 작동했습니다!
▶남은 시간은 60초! 서둘러 방을 탈출하세요!

▶기계 장치 화면에 수식이 적혀있다.
▶[ 50 X 40 + 26 = ? ]
[ ?? 째깍째깍... 폭발까지 남은 시간 : 55초! ]
정답 입력:
```

```
==================================================
                  쾅!!! ??
              시간 초과! 폭탄 폭발!
==================================================


--------------------------------
Process exited after 255 seconds with return value 0
계속하려면 아무 키나 누르십시오 . . .
```

[60초 시한폭탄 카운트다운 - episode2]

```c
#include <time.h> // 시간을 다루기 위한 필수 헤더

// 1. 폭탄이 켜지는 순간의 '현재 시간'을 사진 찍듯 기록해 둡니다.
time_t start_time = time(NULL);

while(1) {
    // 2. 루프가 돌 때마다 '지금 시간'을 계속 확인합니다.
    time_t current_time = time(NULL);

    // 3. (지금 시간) - (시작 시간) = '흘러간 시간'을 구합니다.
    int elapsed_time = (int)(current_time - start_time);

    // 4. 60초에서 흘러간 시간을 빼서 '남은 시간'을 계산합니다!
    int remain_time = 60 - elapsed_time;

    // 5. 남은 시간이 0초 이하가 되면 폭탄이 터집니다.
    if (remain_time <= 0) {
        printf("💥 쾅!! 60초가 지나 폭탄이 터졌습니다!\n");
        break; // 루프 종료
    }
}
```

파일 업그레이드 (5)

```
[ 에피소드 2 : 현재 행동을 선택하세요 ]
[ ?? 째깍째깍... 폭발까지 남은 시간 : 55초! ]
1. 낡은 책장 조사하기
2. 기계 장치 조작하기
3. 아이템 조합하기
4. 어두운 통로로 진입하기 (탈출 시도)
선택:
```

```
[ 에피소드 2 : 현재 행동을 선택하세요 ]
[ ?? 째깍째깍... 폭발까지 남은 시간 : 39초! ]
1. 낡은 책장 조사하기
2. 기계 장치 조작하기
3. 아이템 조합하기
4. 어두운 통로로 진입하기 (탈출 시도)
선택:
```

```
[ 에피소드 2 : 현재 행동을 선택하세요 ]
[ ?? 째깍째깍... 폭발까지 남은 시간 : 06초! ]
1. 낡은 책장 조사하기
2. 기계 장치 조작하기
3. 아이템 조합하기
4. 어두운 통로로 진입하기 (탈출 시도)
선택:
```

[실시간 60초 카운트 화면 표시 - episode2]

```c
while (1) {
    // 남은 시간 계산 (60초 - 흐른 시간)
    int remain = 60 - (int)(time(NULL) - start_time);

      // 3. 타이머가 표시될 위치(맵 왼쪽 0, 지정된 줄 timer_y)로 커서를 순간이동!
      gotoxy(0, timer_y);

      // 4. 그 자리에 남은 시간을 덮어씁니다. (%02d는 '09', '08'처럼 두 자리로 예쁘게 출력)
      printf("[ 💣 째깍째깍... 폭발까지 남은 시간 : %02d초! ]", remain);

      // 5. 방금 숫자를 썼으니 커서가 타이머 옆에 있겠죠?
      // 이걸 다시 플레이어가 글자를 치던 원래 '입력칸 위치'로 재빨리 되돌려 놓습니다!
      gotoxy(input_x, input_y);

      // ========================================

      Sleep(50); // 0.05초 대기 후 다시 위로 올라가서 무한 반복!
    }
}
```

파일 업그레이드 (6)

[3자리 랜덤 수 생성 로직 - episode3]

```
▶[ 전자 금고 해킹 모드 ] - 기회는 단 7번!
▶0~9 사이의 중복되지 않는 숫자 3자리를 맞춰야 합니다.
▶S(자리와 숫자 맞음), B(숫자만 맞음)

(치트키 - 정답: 857)
[남은 기회: 7번] 3자리 숫자 입력:
```

```
▶[ 전자 금고 해킹 모드 ] - 기회는 단 7번!
▶0~9 사이의 중복되지 않는 숫자 3자리를 맞춰야 합니다.
▶S(자리와 숫자 맞음), B(숫자만 맞음)

(치트키 - 정답: 169)
[남은 기회: 7번] 3자리 숫자 입력:
```

```
▶[ 전자 금고 해킹 모드 ] - 기회는 단 7번!
▶0~9 사이의 중복되지 않는 숫자 3자리를 맞춰야 합니다.
▶S(자리와 숫자 맞음), B(숫자만 맞음)

(치트키 - 정답: 152)
[남은 기회: 7번] 3자리 숫자 입력:
```

```
srand((unsigned int)time(NULL));


int ans[3]; // 정답을 저장할 3칸짜리 배열


// 1. 첫 번째 자리 (백의 자리): 1~9 사이의 숫자
// (야구 게임은 보통 0으로 시작하지 않게 만듭니다)
ans[0] = rand() % 9 + 1;


// 2. 두 번째 자리 (십의 자리): 0~9 사이의 숫자
do {
    ans[1] = rand() % 10;
} while (ans[1] == ans[0]); // 앞자리와 같으면 다를 때까지 무한 반복!


// 3. 세 번째 자리 (일의 자리): 0~9 사이의 숫자
do {
    ans[2] = rand() % 10;
} while (ans[2] == ans[0] || ans[2] == ans[1]); // 앞의 두 자리와 겹치면 다시!
```

파일 업그레이드 (7)



[상하좌우 키 로직 - episode3]

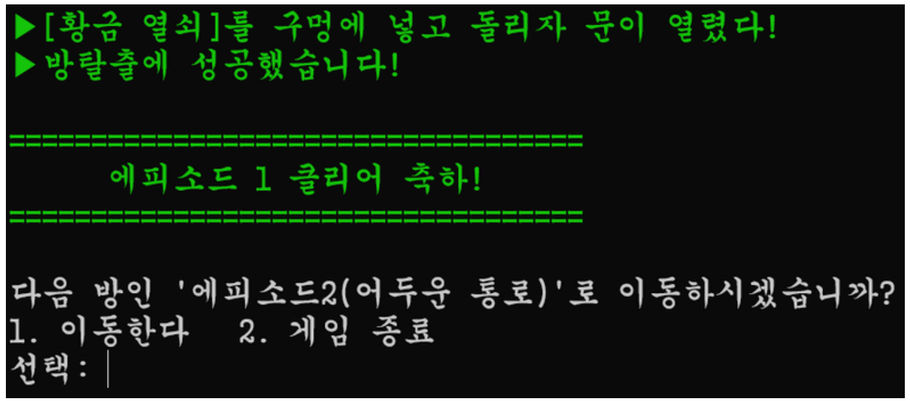
```
// 2. 키보드 입력 받기 (엔터 안 쳐도 바로 인식하는 _getch)
int key = _getch();

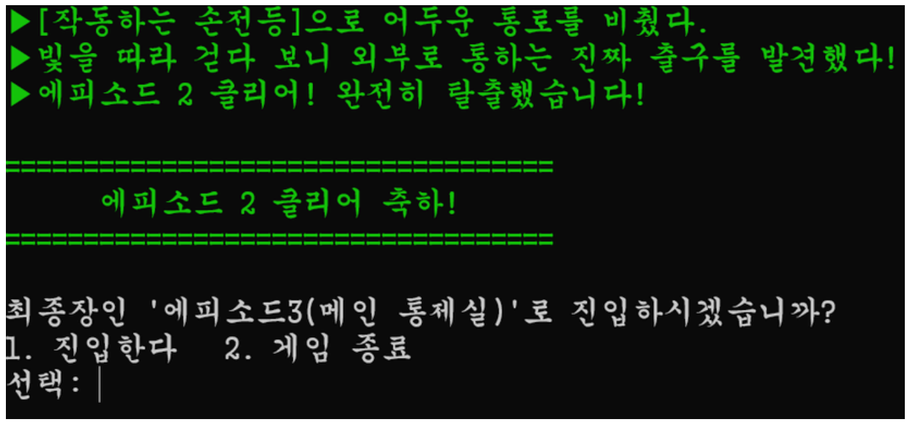
// 만약 입력받은 키가 '방향키 특수 신호(224)'라면?
if (key == 224) {
    key = _getch(); // 진짜 방향키 값을 한 번 더 읽어옴

    // nx, ny는 플레이어가 '이동할 다음 위치'를 미리 계산해 보는 변수입니다.
    int nx = px;
    int ny = py;

    // 3. 입력된 방향에 따라 다음 좌표(nx, ny) 계산
    if (key == 72) ny--;       // ↑ (위로 가면 세로 y가 감소)
    else if (key == 80) ny++; // ↓ (아래로 가면 세로 y가 증가)
    else if (key == 75) nx--; // ← (왼쪽으로 가면 가로 x가 감소)
    else if (key == 77) nx++; // → (오른쪽으로 가면 가로 x가 증가)
```

파일 업그레이드 (8)





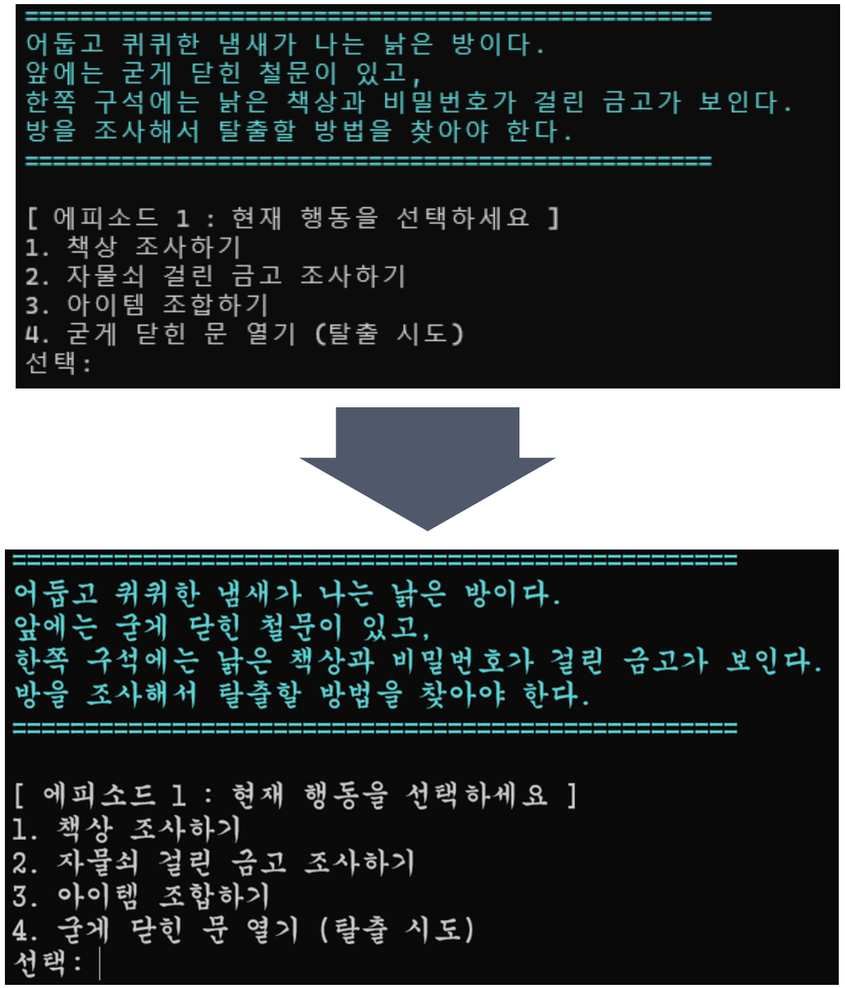
[Next episode 로직]

```c
int ep1_result = play_episode1("에피소드1.txt");

if (ep1_result == 1) { // 에피소드 1에서 return 1(클리어)을 받아왔다면?
    printf("에피소드 1 클리어! 에피소드 2로 이동하시겠습니까? (1.이동 2.종료) : ");
    int next_ep2;
    scanf("%d", &next_ep2);

    if (next_ep2 == 1) {
        system("cls"); // 이동하기 전에 화면을 깨끗하게 지웁니다!
```

파일 업그레이드 (9)



[폰트 및 텍스트 사이즈 변경]

```
void change_font() {
    CONSOLE_FONT_INFOEX cfi;
    cfi.cbSize = sizeof(CONSOLE_FONT_INFOEX);
    GetCurrentConsoleFontEx(GetStdHandle(STD_OUTPUT_HANDLE), FALSE, &cfi);

    cfi.dwFontSize.X = 0;
    cfi.dwFontSize.Y = 24;
    cfi.FontWeight = FW_BOLD;

    wcscpy(cfi.FaceName, L"GungsuhChe");

    SetCurrentConsoleFontEx(GetStdHandle(STD_OUTPUT_HANDLE), FALSE, &cfi);
}
```

총 업그레이드 내용

10 / 30

- 텍스트 색상 변경
- 이벤트 소리 추가
- 상태 관리 및 인벤토리 추가
- 60초 타이머 추가
- 60초 타이머 화면에 표시 추가
- 랜덤 3자리 수 생성 추가
- 입력값을 상하좌우만 받을 수 있는 로직 추가
- 파일을 읽을 조건 추가
- 텍스트 사이즈 조절
- 폰트 변경